

AIR UNIVERSITY ISLAMABAD

NCSA

PROJECT REPORT

Risk Ripple

Subject

Secure Software Design and Development

Instructor

Mam Maila Zahra

Group Lead

Muhammad Bilal

Members

Zain Asif | Faisal Rehman

Abstract

Software security vulnerabilities continue to threaten modern applications because developers often lack fast, understandable, and actionable feedback during the coding phase. Traditional Static Application Security Testing (SAST) tools, while functionally capable, suffer from two widely documented problems: they generate overwhelming volumes of alerts that are difficult to prioritize, and they provide little context about how serious each finding is or what a developer should do about it.

This project proposes Smart SAST -- a lightweight, developer-friendly SAST tool that scans Python source code and addresses these gaps through three novel contributions. First, an **Exploitability Scoring Engine** ranks each finding by a dynamic risk score (0-10) computed from contextual signals, including whether the vulnerable code is reachable from user-controlled input and whether it resides in a public API endpoint. Second, a **Fix-Hint Generator** attaches concise, plain-English remediation guidance directly to every finding, so developers can act immediately without leaving their workflow. Third, a **Noise Filter de-duplicates** repeated findings and suppresses low-impact alerts below a configurable threshold, reducing clutter and helping developers focus on the issues that matter most.

The system uses Python's built-in ast module for structural code parsing, complemented by regex-based pattern rules covering ten vulnerability classes including SQL injection, command injection, hardcoded secrets, insecure deserialization, and weak cryptography. Output is delivered as a structured JSON report and a human-readable HTML summary. The expected contribution is a practical, explainable, and usable scanner tailored for Python codebases, particularly suited to students, small development teams, and DevSecOps workflows where cost, simplicity, and developer experience are primary concerns.

Table of Contents

Abstract.....	2
Table of Contents.....	3
1. Introduction	5
1.1 Background	5
1.2 Importance of the Domain.....	5
1.3 Problem Statement.....	6
1.4 Proposed Solution.....	7
2. Literature Review	8
2.1 Overview of Related Work.....	8
2.2 Summary of Selected Studies	8
Chess & McGraw (2004) -- Static Analysis for Security [1]	8
Johnson et al. (2013) -- Why Don't Developers Use Static Analysis Tools? [2]	8
Sadowski et al. (2018) -- Lessons from Building Static Analysis Tools at Google [3]	9
Ayewah et al. (2008) -- Using Static Analysis to Find Bugs [4].....	9
Livshits & Lam (2005) -- Finding Security Vulnerabilities in Java [5]	9
Arzt et al. (2014) -- FlowDroid: Taint Analysis for Android [6]	9
Smith et al. (2015) -- Questions Developers Ask During SAST Diagnosis [7]	10
Muske & Serebrenik (2016) -- Survey of Approaches for Handling SAST Alarms [8]	10
Christakis & Bird (2016) -- What Developers Want from Program Analysis [9]	10
Viega et al. (2000) -- ITS4: A Lightweight Vulnerability Scanner [10]	11
2.3 Comparison Table of Related Literature.....	11
2.4 Critical Analysis	13
3. Methodology.....	14
3.1 System Architecture.....	14
3.2 Component-Wise Explanation	16
Component 1 -- Repo Loader.....	16
Component 2 -- AST Analyzer	16
Component 3 -- Rule Engine	17
Component 4 -- Exploitability Scorer (Novel Feature 1).....	17
Component 5 -- Fix-Hint Generator (Novel Feature 2).....	18
Component 6 -- Noise Filter (Novel Feature 3).....	18

Component 7 -- Report Generator	19
3.3 Complete End-to-End Workflow	19
4. Use Case Diagram and Descriptions	20
4.1 System Actors	20
4.2 Use Case Diagram	20
4.3 Use Case Descriptions	21
UC-1: Scan Repository.....	21
UC-2: View Security Report.....	21
UC-3: Configure Noise Filter	22
UC-4: Integrate into CI/CD Pipeline	22
5. Test Cases.....	23
5.1 Test Case Notes.....	25
6. Market Value and Relevance	26
6.1 Target Users	26
6.2 Market Relevance	26
6.3 Competitive Analysis.....	27
7. Gantt Chart -- Project Timeline	29
7.1 Timeline Notes	30
8. Results and Conclusion	31
8.1 Expected Outcomes	31
8.2 Planning Phase Summary.....	Error! Bookmark not defined.
8.3 Conclusion	32
References	33
Appendix	35
Appendix A: Complete Vulnerability Rule Reference	35
Appendix B: Sample Vulnerable Python Test Fixture	Error! Bookmark not defined.
Appendix C: JSON Report Output Schema.....	Error! Bookmark not defined.
Appendix D: Glossary of Technical Terms.....	36

1. Introduction

1.1 Background

Software vulnerabilities have been a persistent challenge throughout the history of computing. As software systems grow in complexity, lines of code multiply, and development teams expand, the probability of introducing security flaws increases proportionally. A single undetected vulnerability in a production system can result in unauthorized data access, financial loss, regulatory penalties, and long-term reputational damage. According to the National Vulnerability Database (NVD), over 25,000 Common Vulnerabilities and Exposures (CVEs) were officially published in 2023 alone, reflecting a relentless and growing threat surface.

Static Application Security Testing (SAST) is an established approach to detecting security flaws in source code before the software is executed or deployed. Unlike dynamic testing methods that require a running application environment, SAST tools analyze code structure, control flow, and data patterns at the source level. This makes SAST particularly valuable in early stages of the Software Development Life Cycle (SDLC), where fixing a bug costs significantly less -- studies estimate 10 to 100 times less -- than addressing it after deployment.

Python has become one of the most widely used programming languages globally, powering web applications (Flask, Django), data science pipelines, automation scripts, and machine learning systems. Its flexible, interpreted nature, while highly productive, introduces security risks that static analysis tools must handle specifically. Despite Python's widespread adoption, Python-specific SAST tooling lags behind Java and C/C++ ecosystems in terms of precision, developer experience, and fix guidance.

1.2 Importance of the Domain

Secure software design is increasingly a legal, regulatory, and business obligation. Standards such as OWASP Top 10, NIST Secure Software Development Framework (SSDF - SP 800-218), and the European Union's Cyber Resilience Act (2024) mandate that organizations embed security into their development processes. The US Executive Order on Improving the Nation's Cybersecurity (EO 14028, May 2021) specifically calls for adoption of security testing tools in software supply chains. SAST tools are a primary technical mechanism for meeting these requirements without imposing significant workflow overhead on developers.

Research consistently shows that when SAST tools produce results developers can understand and act upon, they substantially reduce the number of vulnerabilities reaching production. Conversely, tools that generate excessive noise or unhelpful output are frequently disabled by developers, rendering the security investment ineffective. The usability gap between SAST tool capability and real-world developer adoption is therefore not only technically significant -- it carries direct security consequences for organizations.

1.3 Problem Statement

Existing SAST tools for Python source code suffer from three interconnected problems that limit their practical usefulness:

- **Noisy Output:** Tools such as Bandit can generate hundreds of findings for a moderately-sized project, many of which are low-confidence or low-impact issues that do not represent real threats. Developers experience alert fatigue and begin ignoring all output uniformly, including genuine critical findings.
- **Lack of Contextual Prioritization:** Most tools assign a static severity label (High / Medium / Low) based only on the vulnerability type, without considering execution context. A hardcoded secret in unreachable dead code and one embedded in a live public API endpoint both receive the same label, despite dramatically different risk levels.
- **Absent Fix Guidance:** Most tools identify what is wrong but not how to fix it. Developers must interrupt their workflow to search external documentation, which increases remediation time and reduces the probability of correct and complete fixes.

Together, these three problems mean developers receive a long list of flags with no clear indication of what to fix first or how to fix it. This systematically undermines the purpose of SAST within the development workflow.

1.4 Proposed Solution

Smart SAST is a lightweight, command-line SAST tool written in Python that directly addresses the three problems above through three corresponding novel features:

- **Exploitability Scoring Engine:** Instead of static severity labels, Smart SAST computes a dynamic risk score (0.0 to 10.0) for each finding based on contextual signals including: whether vulnerable code is reachable from user-controlled input (taint), whether it resides in a public API route (exposure), and the inherent severity of the vulnerability class. This produces a differentiated, context-aware risk ranking.
- **Fix-Hint Generator:** Every finding is accompanied by a short, plain-English remediation hint written specifically for that vulnerability class. Each hint states both what to avoid and what to use instead, including a concrete replacement code example. This allows developers to act immediately without switching context.
- **Noise Filter:** A de-duplication and threshold suppression layer reduces output clutter by fingerprinting and collapsing repeated findings, and hiding findings below a configurable minimum score. Developers control the detail level; suppressed findings are counted but not listed in the main report.

The system accepts a repository path via command-line interface, analyzes all Python (.py) files, and produces a findings.json and report.html. It is designed for fast local execution with no internet dependency, making it suitable for student projects, startup development teams, and CI/CD pipeline integration.

2. Literature Review

2.1 Overview of Related Work

The domains of static analysis, software security testing, and security usability have been actively researched for over two decades. This chapter reviews ten carefully selected research works that are most directly relevant to the Smart SAST project. Papers were selected from IEEE Xplore, ACM Digital Library, and USENIX proceedings based on their relevance to: static analysis for vulnerability detection, false positive reduction, developer experience with SAST tools, taint tracking models, and risk scoring. Each selected paper contributes either a technical insight that shapes Smart SAST's design or an empirical finding that validates its motivation.

2.2 Summary of Selected Studies

Chess & McGraw (2004) -- Static Analysis for Security [1]

This foundational paper makes the case for static analysis as a security discipline. The authors argue that source code analysis -- examining code structure without executing it -- can detect broad classes of vulnerabilities including injection flaws, buffer overflows, and race conditions before the software is deployed. They introduce the idea that SAST should be treated as a core engineering activity integrated into the development workflow, not a post-hoc review step. This philosophical alignment directly mirrors Smart SAST's goal of lightweight, developer-integrated scanning. The paper also identifies the challenge of false positives as the primary adoption barrier -- a problem that Smart SAST's Noise Filter directly addresses.

Johnson et al. (2013) -- Why Don't Developers Use Static Analysis Tools? [2]

This is one of the most frequently cited papers on SAST usability. Through semi-structured interviews with 20 professional developers at large software organizations, the authors identify the primary barriers to SAST adoption: too many false positives, overwhelming output volume, difficulty interpreting results, and findings that arrive too late in the cycle to be acted upon cheaply. Critically, the study finds that developers do not avoid SAST tools because they are indifferent to security -- they stop using tools that make it hard to distinguish important findings from noise. Every one of these barriers is addressed by a specific component of Smart SAST: the Noise Filter (volume), the Exploitability Scorer (interpretation and prioritization), and the Fix-Hint Generator (actionability).

Sadowski et al. (2018) -- Lessons from Building Static Analysis Tools at Google [3]

The Google Tricorder paper is a landmark study in industrial SAST deployment at scale. The authors describe how Google achieved high developer engagement with static analysis by enforcing three rules: only show findings the analyzer is confident are real issues, provide fix suggestions directly alongside each finding, and integrate analysis into the code review workflow rather than as a separate report. They report that even marginal reductions in false positive rates, combined with inline fix suggestions, produced a dramatic increase in the percentage of findings that developers actually acted upon. Smart SAST's Fix-Hint Generator and Noise Filter are directly and explicitly inspired by these findings. The paper is the strongest empirical validation that the three novel features of Smart SAST address the correct problems.

Ayewah et al. (2008) -- Using Static Analysis to Find Bugs [4]

The FindBugs paper demonstrates that pattern-based static analysis can scale to large codebases and detect meaningful bugs at low computational cost. The authors describe a bytecode-level pattern library covering hundreds of bug types. Smart SAST adopts this scalable pattern philosophy using Python's AST module instead of bytecode, and extends it with security-specific rule semantics and contextual risk scoring that FindBugs does not provide. The paper's discussion of the tradeoff between detection completeness and false positive rate informs the calibration of Smart SAST's base severity scores.

Livshits & Lam (2005) -- Finding Security Vulnerabilities in Java [5]

This paper introduces taint analysis as a method for detecting web application vulnerabilities including SQL injection and Cross-Site Scripting. The core concept is that a vulnerability exists when untrusted input (a source, such as a request parameter) reaches a sensitive function (a sink, such as a SQL execute call) without passing through a sanitization step. Smart SAST implements a simplified version of this source-to-sink tracking within individual function bodies (intra-procedural taint analysis), which is sufficient to detect the most common injection patterns while keeping computational cost low. The taint detection result directly feeds into the Exploitability Scorer's taint bonus (+2 points).

Arzt et al. (2014) -- FlowDroid: Taint Analysis for Android [6]

FlowDroid advances taint analysis to handle the complex, event-driven lifecycle of Android applications with high precision and recall. Its key contribution -- context-sensitivity in tracking data flows -- is conceptually central to Smart SAST's Exploitability Scoring model. The idea that

the same vulnerability pattern carries different risk depending on where it appears in the program (inside an event handler vs. inside a utility function) maps directly to Smart SAST's exposure bonus, which adds risk weight to findings inside web route handlers compared to those in internal or test code.

Smith et al. (2015) -- Questions Developers Ask During SAST Diagnosis [7]

Using a think-aloud laboratory study with 12 professional developers, this paper identifies the precise questions developers ask when presented with a SAST finding: Is this actually a bug? How serious is it? How do I fix it? These three questions define the informational needs that Smart SAST is designed to meet. The Exploitability Scorer answers 'Is this serious?', the Fix-Hint Generator answers 'How do I fix it?', and the Noise Filter ensures developers only see findings where 'Is this actually a bug?' is likely to be answered yes. This paper provides the clearest empirical grounding for Smart SAST's design decisions.

Muske & Serebrenik (2016) -- Survey of Approaches for Handling SAST Alarms [8]

This systematic survey reviews 74 papers on techniques for managing static analysis alarms, cataloguing approaches including alarm classification, machine learning-based filtering, clustering, history-based suppression, and threshold gating. The survey finds that de-duplication and rule-based suppression are the most effective low-overhead techniques for small-to-medium scale deployments, validating the approach taken by Smart SAST's Noise Filter. Approaches requiring historical data or ML training are not feasible for the lightweight, first-run scenario that Smart SAST targets.

Christakis & Bird (2016) -- What Developers Want from Program Analysis [9]

A survey of 375 developers at Microsoft quantitatively establishes priorities for program analysis tools. The top three developer needs are: catching real bugs (97%), providing fix suggestions (91%), and explaining why a result is a problem (88%). This data directly validates the Fix-Hint Generator as the single highest-impact feature for developer adoption of Smart SAST. The paper also finds that developers consistently overestimate how well current tools meet their needs, suggesting that even partial improvement in fix guidance creates measurable satisfaction gains.

Viega et al. (2000) -- ITS4: A Lightweight Vulnerability Scanner [10]

ITS4 is an early but influential lightweight SAST tool that uses simple lexical pattern matching to flag calls to dangerous functions in C code. Despite its simplicity, the tool found real vulnerabilities in widely-used open-source software. Smart SAST inherits the lightweight philosophy of ITS4 -- fast, simple, no complex setup -- while upgrading from lexical matching to AST-level analysis, adding contextual scoring, and including fix hints. ITS4 demonstrates that effective security tools do not need to be complex; they need to be accurate enough and fast enough to be used regularly.

2.3 Comparison Table of Related Literature

Table 1: Comparative Analysis of Selected Research Papers

ID	Reference	Method / Approach	Dataset / Target	Tools	Pros	Cons
[1]	Chess & McGraw (2004), IEEE Security & Privacy, 2(6), 76-79.	Source code pattern matching + dataflow analysis	C and C++ programs	Fortify SCA (prototype)	Demonstrates SAST can catch real vulns before deployment; foundational framework	High false positive rate; no developer-facing fix guidance; C/C++ only
[2]	Johnson et al. (2013), ICSE 2013, pp. 672-681.	Qualitative study: interviews with 20 developers at large software firms	Developer survey + existing SAST tool logs	FindBugs, PMD, Coverity	Identifies key SAST usability barriers with high practical relevance; widely cited	May not generalize beyond large enterprise developers; lab vs. real-world gap
[3]	Sadowski et al. (2018), CACM, 61(4), 58-66.	Large-scale incremental SAST deployment at Google; developer feedback loop	Millions of lines of Google production code	Tricorder, Error Prone, Clang-Tidy	Noise filtering + fix suggestions dramatically increase developer engagement with findings	Google-scale infrastructure not reproducible for small teams or academic settings
[4]	Ayewah et al. (2008), IEEE Software, 25(5), 22-29.	Bytecode-level pattern-based bug detection	Java applications including Google codebase subsets	FindBugs	Shows pattern-based SAST scales to large Java codebases; open-source model	Java-focused; most findings are non-security bugs; no risk ranking or fix guidance

ID	Reference	Method / Approach	Dataset Target /	Tools	Pros	Cons
[5]	Livshits & Lam (2005), USENIX Security Symposium, pp. 271-286.	Context-sensitive taint analysis tracking user input to dangerous sinks	Java web applications (servlets, JSPs)	WebSSARI (custom)	Introduces taint tracking for XSS and SQL injection detection with high precision	Computationally expensive; Java-specific; does not address developer experience
[6]	Arzt et al. (2014), ACM SIGPLAN Notices, 49(6), 259-269.	Context/flow/field/object-sensitive taint analysis with lifecycle awareness	500 Android apps from Google Play and malware repos	FlowDroid (Soot-based)	Highly precise taint tracking; context-sensitivity reduces false positives markedly	Android-specific and complex; not directly applicable to Python web applications
[7]	Smith et al. (2015), ESEC/FSE 2015, pp. 248-259.	Lab study: think-aloud protocol observing developers diagnosing SAST findings	12 professional developers; Java apps; real CVE-related findings	FindBugs + commercial tool	Reveals exact cognitive questions developers need answered (severity? fix? real bug?)	Small sample; lab setting may not reflect production time pressure
[8]	Muske & Serebrenik (2016), SCAM 2016, pp. 157-166.	Systematic survey of 74 papers on alarm handling in static analysis	Literature survey (secondary research)	Multiple tools surveyed	Comprehensive taxonomy of noise-reduction techniques; validates dedup and suppression	Survey-based; no primary empirical evaluation; somewhat dated (2016)
[9]	Christakis & Bird (2016), ASE 2016, pp. 332-343.	Survey of 375 Microsoft developers on program analysis tool needs	Developer survey at Microsoft	PREfix, Code Contracts, Coverity	91% of developers cited fix suggestions as important; validates Fix-Hint Generator	Single large organization; findings may differ for students or small teams
[10]	Viega et al. (2000), ACSAC 2000, pp. 257-267.	Lexical pattern matching against dangerous function signatures	Open-source C and C++ projects	ITS4 (custom lexical scanner)	Early proof that lightweight SAST is feasible; fast and simple approach	Lexical only -- no dataflow or context; very high false positives; C/C++ only

2.4 Critical Analysis

A consistent theme emerges across all ten reviewed papers: the technical capability of static analysis tools has outpaced their practical usability. Livshits & Lam [5] and Arzt et al. [6] demonstrate that taint analysis achieves high precision, but neither addresses what happens after a finding is produced -- whether the developer can understand it and what they should do. Chess & McGraw [1] and Viega et al. [10] show that even simple pattern matching identifies real vulnerabilities, but raw results without prioritization overwhelm developers.

The usability-focused papers [2, 3, 7, 9] collectively establish that SAST adoption failure is a tool design problem, not a developer attitude problem. Developers want to find and fix security bugs; they stop using tools that create more work than they eliminate. Sadowski et al.'s [3] experience at Google is the most instructive data point: noise reduction combined with inline fix suggestions produced the largest measurable improvement in developer engagement with static analysis findings.

The survey by Muske & Serebrenik [8] provides technical validation for the Noise Filter approach. Notably, most high-performing techniques surveyed require historical data or machine learning -- methods not feasible for a lightweight first-run tool. Deterministic de-duplication and threshold suppression, as implemented in Smart SAST, are validated by the survey as effective and practical alternatives.

A clear gap in the literature is the absence of a tool that combines exploitability-aware scoring, built-in fix hints, and intelligent noise filtering in a single lightweight package accessible to student and small-team developers. Enterprise tools such as Veracode and Checkmarx include subsets of these features but require costly licenses and cloud infrastructure. Open-source tools such as Bandit and Semgrep address detection but not prioritization or fix guidance in an integrated manner. Smart SAST targets this specific and underserved gap in the tooling landscape.

3. Methodology

3.1 System Architecture

Smart SAST follows a seven-stage pipeline architecture in which Python source files pass through a sequence of processing stages, each adding analysis depth, contextual enrichment, or output filtering before the final report is generated. The diagram below illustrates the complete data flow from user input through to report output.

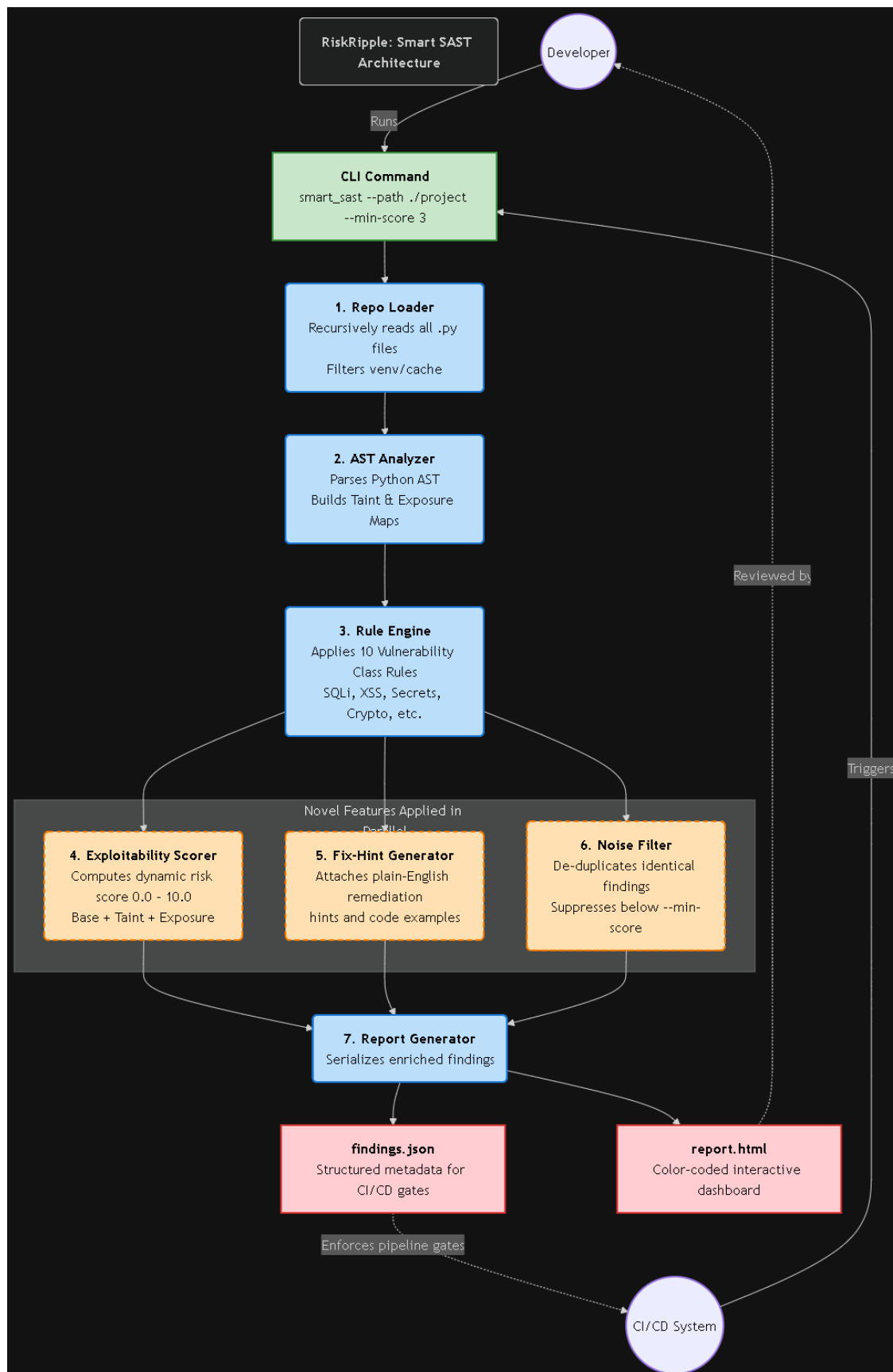


Figure 1: Smart SAST System Architecture -- Data Flow Diagram

The pipeline is designed around a single principle: each stage should add only what subsequent stages need, and nothing more. This keeps the system modular, testable in isolation, and easy to extend with new vulnerability rules or output formats.

3.2 Component-Wise Explanation

Component 1 -- Repo Loader

The Repo Loader is the entry point of the system. It receives a directory path provided by the user via the argparse-based command-line interface. It then walks the directory tree recursively using Python's `os.walk()` function and collects all files with a `.py` extension. The loader filters out virtual environment directories (`venv/`, `.venv/`, `env/`), build artifacts (`__pycache__`), and optionally test directories based on a flag. Each collected file is returned as an ordered list of absolute paths for downstream processing. The loader also reports the total file count and aggregate line count as part of the scan summary.

Component 2 -- AST Analyzer

The AST Analyzer reads each Python file and parses it into an Abstract Syntax Tree (AST) using Python's built-in `ast` module. An AST is a hierarchical tree representation of the grammatical structure of source code. For example, the statement `eval(user_input)` becomes a `Call` node in the AST, with a `Name` node (`eval`) as the function and a `Name` node (`user_input`) as the argument. The Analyzer traverses these trees using the `ast.NodeVisitor` pattern, building two auxiliary data structures that later stages use:

- **Taint Map:** For each function in the file, the Analyzer identifies which local variables may carry user-controlled data. A variable is tainted if it is assigned from a function parameter (potential user input) or from a known taint source (e.g., `request.args.get()`, `input()`, `sys.stdin.read()`). The taint map tracks how tainted values flow through local assignments within the function.
- **Exposure Map:** The Analyzer checks whether each function is decorated with a web route decorator from Flask (`@app.route`), Django (`path()`, `re_path()`), or FastAPI (`@router.get()` etc.). Functions with such decorators are marked as exposed, indicating they are publicly reachable from network requests.

Component 3 -- Rule Engine

The Rule Engine applies a library of security rules against the AST trees and taint data produced by the Analyzer. Each rule is an independent Python class implementing a standard interface: it receives an AST node, the taint map, the exposure map, and the file path, and returns a Finding object if the node matches an insecure pattern. Rules are organized into two categories:

- **AST Rules:** Structural checks that examine the AST directly. Examples include detecting calls to `eval()`, `exec()`, `compile()`, `pickle.loads()`, `subprocess` with `shell=True`, or `yaml.load()` without a safe Loader argument.
- **Regex Rules:** Pattern checks applied to string literal values in the AST. Examples include detecting hardcoded passwords, API keys, and tokens in string assignments, weak cryptographic algorithm names (MD5, SHA1, DES), and debug mode flags.

The ten vulnerability classes covered by the Rule Engine are: SQL Injection, Cross-Site Scripting (XSS), Hardcoded Secrets, Insecure Deserialization, Weak Cryptography, Command Injection, Path Traversal, Insecure Randomness, Debug/Logging Leaks, and Unsafe File Handling.

Component 4 -- Exploitability Scorer (Novel Feature 1)

The Exploitability Scorer computes a numerical risk score from 0.0 to 10.0 for each raw Finding produced by the Rule Engine. The score is computed using a weighted additive formula with four components:

Scoring Component	Points	Description
Base Severity	1.0 - 4.0	Assigned per vulnerability class based on intrinsic risk (e.g., Command Injection = 4.0, Insecure Randomness = 1.5)
Taint Bonus	+2.0	Added when the Analyzer's taint map confirms user-controlled data flows to the finding's sink within the function
Exposure Bonus	+2.0	Added when the finding is inside a function marked as exposed (a public web route decorator detected)
Context Penalty	-1.0	Subtracted when the finding is inside a test file (path contains <code>/test/</code> or <code>_test.py</code>), a private function (name starts with <code>_</code>), or a comment-marked safe block

Table 2: Exploitability Score Formula Components

The final score is clamped to the range [0.0, 10.0]. Findings are labeled: Critical (8.0-10.0), High (6.0-7.9), Medium (4.0-5.9), Low (2.0-3.9), or Informational (0.0-1.9). Because the score is computed per-finding with context, two findings of the same vulnerability type -- say, SQL

injection -- can receive very different scores if one is in an API endpoint with tainted input (score: ~8.5) and the other is in a test utility with no external callers (score: ~3.5).

Component 5 -- Fix-Hint Generator (Novel Feature 2)

The Fix-Hint Generator is a deterministic lookup module that attaches a short, plain-English remediation string to every finding based on its rule ID. Each hint has two parts: what the insecure pattern is, and what the secure alternative is, accompanied by a concrete replacement code example. Fix hints are stored as a static dictionary keyed by rule_id and are loaded once at startup. Every rule in the Rule Engine has a corresponding entry in the fix hint dictionary -- 100% coverage is guaranteed by design.

Example fix hints for selected rules:

- sqli-001 (SQL Injection): 'Avoid building SQL queries by string concatenation. Use parameterized queries with placeholders: `cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))`'
- crypto-001 (Weak Crypto): 'Replace MD5 or SHA-1 with SHA-256 or SHA-3. Use: `hashlib.sha256(data).hexdigest()`. Never use MD5 or SHA-1 for password hashing -- use `bcrypt` or `argon2` instead.'
- secret-001 (Hardcoded Secret): 'Remove credentials from source code. Load secrets from environment variables: `os.environ.get("SECRET_KEY")` or use a dedicated secrets manager such as AWS Secrets Manager or HashiCorp Vault.'
- cmdi-001 (Command Injection): 'Pass commands as a list, not a string, and set `shell=False`: `subprocess.run(["ls", "-la"], shell=False)`. Never construct shell commands from user input.'

Component 6 -- Noise Filter (Novel Feature 3)

The Noise Filter reduces output clutter through two distinct mechanisms applied sequentially. First, de-duplication: each finding is fingerprinted by computing a hash of the tuple (file_path, rule_id, line_number, code_snippet). Findings with identical fingerprints -- which arise when the same vulnerable pattern appears multiple times in a repeated code block or copy-pasted section -- are merged into a single entry. The merged entry carries a repeat_count field indicating how many times the pattern was detected. Second, threshold suppression: findings with a risk score below the configurable --min-score threshold (default: 2.0) are moved from the main findings list to a suppressed list. The main report does not show suppressed findings individually but includes a count in the summary line (e.g., '3 findings suppressed below threshold 2.0').

Component 7 -- Report Generator

The Report Generator serializes the processed, enriched, and filtered list of findings into two output formats. The JSON report (findings.json) is a structured file containing all finding metadata: file, line, code snippet, rule ID, risk score, severity label, fix hint, taint status, exposure status, repeat count, and suppressed status. The HTML report (report.html) is a styled, self-contained web page (no external CDN dependencies) that renders findings in a color-coded severity table, sorted by risk score descending. Each row in the HTML table includes an expandable section showing the code snippet with syntax highlighting and the fix hint. A summary dashboard at the top of the HTML report shows counts by severity level.

3.3 Complete End-to-End Workflow

1. Developer invokes the tool: `smart_sast --path ./my_project --min-score 3 --output ./results`
2. Repo Loader walks the directory, collects .py files, filters out venv and cache directories
3. For each file: AST Analyzer parses the source into an AST tree and builds taint and exposure maps
4. Rule Engine applies all 10 vulnerability class rules to each AST, producing a list of raw Finding objects
5. Exploitability Scorer computes a risk score (0-10) for each Finding using the base + taint + exposure formula
6. Fix-Hint Generator looks up the fix hint for each Finding's rule_id and attaches it to the Finding
7. Noise Filter fingerprints and de-duplicates Findings, then applies threshold suppression
8. Report Generator writes findings.json and report.html to the output directory
9. System prints a scan summary: 'Scan complete: 9,241 lines in 23 files | 15 findings (2 Critical, 4 High, 6 Medium, 3 Low) | 7 suppressed'
10. Developer opens report.html in a browser, reviews findings sorted by risk score, and uses fix hints for remediation

4. Use Case Diagram and Descriptions

4.1 System Actors

The Smart SAST system involves two primary actors and the system itself:

- **Developer:** The primary user who invokes the tool from the command line, reviews the generated report, and applies the fix hints to their code. The developer may be a university student, a junior software engineer, or a senior DevSecOps practitioner. They interact with the system directly through the CLI and browser-rendered HTML report.
- **CI/CD System:** An automated build and deployment pipeline (such as GitHub Actions, GitLab CI, or Jenkins) that invokes Smart SAST as part of a pipeline step during pull request checks or pre-merge gates. The CI/CD system consumes the JSON output programmatically and uses the tool's exit code to enforce quality gates.

4.2 Use Case Diagram

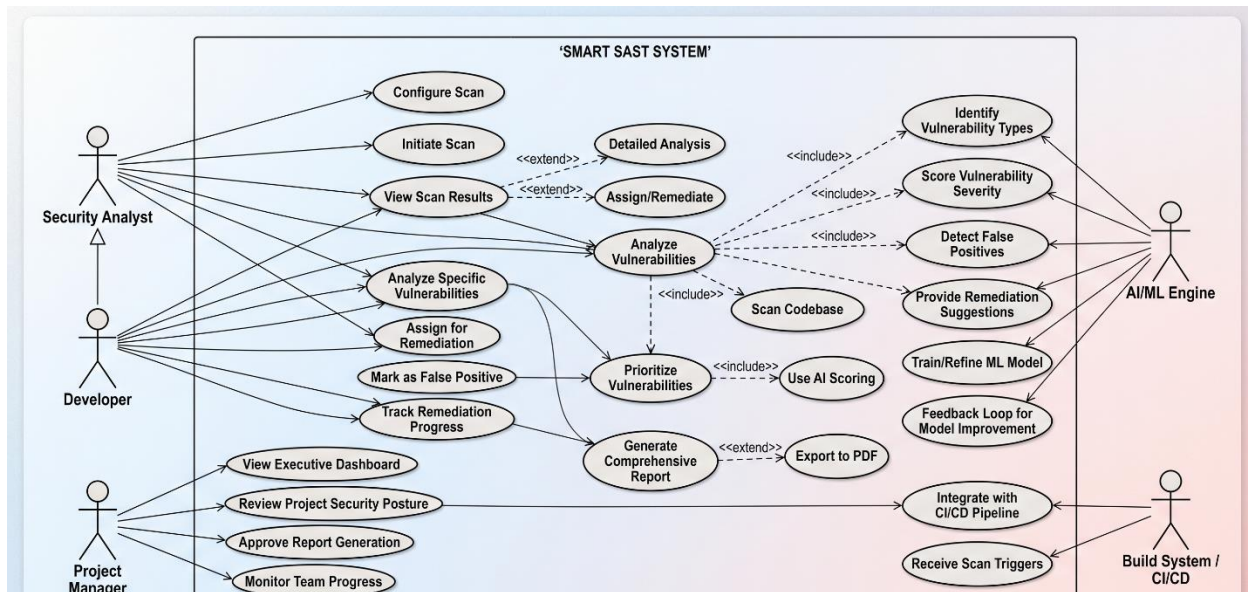


Figure 2: Use Case Diagram -- Smart SAST System

4.3 Use Case Descriptions

UC-1: Scan Repository

Use Case ID	UC-1
Name	Scan Repository
Actors	Developer
Preconditions	Python source files exist at the target path. Smart SAST is installed (pip install smart-sast). The developer has read access to the target directory.
Main Flow	1. Developer opens terminal and navigates to project root. 2. Developer runs: <code>smart_sast --path ./src</code> . 3. Repo Loader collects all .py files. 4. AST Analyzer parses each file. 5. Rule Engine applies all 10 vulnerability class rules. 6. Exploitability Scorer computes a risk score for each Finding. 7. Fix-Hint Generator attaches the appropriate remediation hint. 8. Noise Filter de-duplicates and applies threshold suppression. 9. Report Generator writes findings.json and report.html. 10. CLI prints summary: 'Scan complete. 15 findings 2 Critical 4 High 6 Medium 3 Low'.
Postconditions	findings.json and report.html are produced in the output directory. Each finding includes a risk score, severity label, fix hint, and code snippet. Exit code is 0 unless -fail-on-critical is set and Critical findings exist.

UC-2: View Security Report

Use Case ID	UC-2
Name	View Security Report
Actors	Developer
Preconditions	A scan has been completed and report.html exists. The developer has a web browser available.
Main Flow	1. Developer opens report.html in a web browser (no internet connection required). 2. Report displays a summary dashboard with findings by severity (color-coded). 3. Findings are listed sorted by risk score descending (Critical first). 4. Developer clicks on a Critical or High severity finding row. 5. Row expands to show: file path, line number, code snippet, exploitability score breakdown, and fix hint. 6. Developer reads the fix hint, modifies the code, and saves the file. 7. Developer re-runs <code>smart_sast</code> to confirm the finding is resolved.
Postconditions	Developer has a clear, actionable view of all findings. Critical and High findings are immediately visible. The developer can start remediation directly from the fix hint without needing to consult external documentation.

UC-3: Configure Noise Filter

Use Case ID	UC-3
Name	Configure Noise Filter
Actors	Developer
Preconditions	Developer is familiar with the codebase and wants to focus on high-priority findings. A previous scan produced too many low-severity results.
Main Flow	1. Developer reviews summary from initial scan and decides to suppress Low and Informational findings. 2. Developer re-runs: <code>smart_sast --path ./src --min-score 4</code> . 3. Noise Filter applies the threshold: all findings with score < 4.0 are moved to the suppressed list. 4. Main report shows only Medium, High, and Critical findings. 5. Summary line reads: 'Showing 9 findings 6 suppressed below threshold 4.0'. 6. Developer reviews only the high-priority findings without distraction from low-impact noise.
Postconditions	Output report contains only findings at or above the specified threshold. Suppressed findings are counted in the summary but not listed individually in the main report. Developer spends less time triaging and more time remediating real issues.

UC-4: Integrate into CI/CD Pipeline

Use Case ID	UC-4
Name	Integrate into CI/CD Pipeline
Actors	CI/CD System, Developer
Preconditions	Smart SAST is installable via pip in the CI/CD runner environment. A pipeline configuration file (e.g., <code>.github/workflows/sast.yml</code>) has been set up. The team has agreed on an acceptable risk threshold for blocking merges.
Main Flow	1. Developer pushes a feature branch and opens a pull request. 2. CI/CD pipeline triggers automatically on the pull request event. 3. Pipeline installs Smart SAST: <code>pip install smart-sast</code> . 4. Pipeline step runs: <code>smart_sast --path ./src --min-score 6 --fail-on-critical</code> . 5. Smart SAST scans the code and writes <code>findings.json</code> . 6. If any finding has score ≥ 8.0 (Critical), the process exits with code 1. 7. CI/CD system interprets exit code 1 as a pipeline failure and blocks the pull request merge. 8. Developer receives a notification, reviews the report, and resolves Critical findings. 9. After fixing, developer re-pushes; pipeline re-runs and exits with 0; merge is unblocked.
Postconditions	Pull requests containing Critical-severity findings are blocked from merging until resolved. The JSON artifact provides traceable evidence of security review for.

5. Test Cases

The following test cases cover the complete functional scope of Smart SAST including vulnerability detection accuracy, all three novel feature behaviors, edge cases, and negative tests. Each test case can be implemented as a pytest test function in the project's test suite. Test cases TC-01 through TC-05 validate detection and scoring; TC-03 and TC-06 validate the Noise Filter; TC-08 validates the absence of false positives.

Table 3: Smart SAST Test Case Specifications

TC ID	Description	Input	Expected Output	Feature Tested
TC-01	SQL Injection via String Concatenation in Flask Route	File: api.py @app.route('/user') def get_user(): uid=request.args.get('id'); db.execute('SELECT * FROM users WHERE id='+uid)	Finding: SQL Injection (sqli-001) Score >= 8.0 (taint bonus + exposure bonus applied) Severity: Critical Fix hint mentions parameterized queries Taint: true Exposure: public_api_route	Rule Engine + Taint Tracking + Exploitability Scorer + Fix-Hint Generator
TC-02	Hardcoded API Secret in Source File	File: config.py API_KEY = 'sk-abc123supersecretkey9999xyz' No decorators or user input	Finding: Hardcoded Secret (secret-001) Score: 3.0 (base only, no taint/exposure) Severity: Low-Medium Fix hint mentions os.environ.get() Taint: false Exposure: none	Regex Rule + Fix-Hint Generator
TC-03	Noise Filter: De-duplication of Repeated Pattern	File: utils.py 10 identical instances of pickle.loads(data) spread across different functions	Output contains exactly 1 finding for pickle.loads repeat_count: 10 Total raw findings (10) collapsed to 1 de-duplicated finding Noise reduction: 90%	Noise Filter -- De-duplication
TC-04	Weak Cryptography --	File: auth.py import hashlib; hashed = hashlib.md5(password.encode()).hexdigest()	Finding: Weak Cryptography (crypto-001)	AST Rule + Fix-Hint Generator

TC ID	Description	Input	Expected Output	Feature Tested
	MD5 Password Hashing		Score: 2.5-4.5 (base severity, no taint/exposure) Severity: Low-Medium Fix hint recommends SHA-256 or bcrypt for passwords	+ Exploitability Scorer
TC-05	Command Injection via subprocess with shell=True in API Endpoint	File: runner.py @app.route('/run') def run(): cmd=request.args.get('cmd'); subprocess.run(cmd, shell=True)	Finding: Command Injection (cmdi-001) Score >= 9.0 (taint: +2, exposure: +2, base: ~4) Severity: Critical Fix hint: use shell=False and arg list Exit code 1 if --fail-on-critical set	AST Rule + Taint + Exposure + Exploitability Scorer + CLI Exit Code
TC-06	Threshold Suppression -- Low Score Finding Hidden	File: animation.py x = random.random() # used for visual animation only Tool invoked with: --min-score 4	Finding: Insecure Randomness (rand-001) score < 4.0 Finding NOT in main report Summary shows '1 finding suppressed below threshold 4.0' Main report is empty Exit code: 0	Noise Filter -- Threshold Suppression + Min-Score CLI Flag
TC-07	Flask Debug Mode Enabled in Production File	File: app.py (main application file, not test) app.run(debug=True) at end of file	Finding: Debug/Logging Leak (debug-001) Score: 5.0-7.0 (exposure bonus if in app file) Severity: Medium-High Fix hint: set debug=False in production or use env variable	Regex Rule + Exploitability Scorer + Fix-Hint Generator
TC-08	Clean File -- Zero Findings (True Negative Test)	File: math_utils.py Pure arithmetic functions, no user input, no dangerous calls, no hardcoded strings	Zero findings for this file JSON output: {findings: [], summary: {total: 0}} HTML report: 'No security issues detected' CLI exits with code 0 No	Negative Test -- Correct Silence / False Positive Prevention

TC ID	Description	Input	Expected Output	Feature Tested
			false positives raised	

5.1 Test Case Notes

- All test cases will be implemented using pytest with fixture-based test Python files created specifically to trigger each vulnerability pattern.
- TC-01 and TC-05 are the most critical tests as they validate the integration of taint tracking, exposure detection, and scoring in a single pipeline.
- TC-08 (clean file test) is equally important -- false positives erode developer trust. A tool that reports zero issues on a clean file is as important as one that catches real issues.
- Additional edge case tests will be added during implementation for: empty files, files with only comments, files with encoding errors, and very large files (> 5,000 lines).

6. Market Value and Relevance

6.1 Target Users

Smart SAST is designed to serve three distinct user groups, each of which is currently underserved by existing tooling:

- **University Students and Academic Teams:** Computer science and software engineering students learning secure coding practices need simple, fast, local tools that produce understandable output. Smart SAST is designed to run with a single command, requires no cloud account or license key, and produces reports that explain not just what is wrong but why and how to fix it -- making it ideal for coursework, capstone projects, and security labs at institutions like Air University Islamabad.
- **Solo Developers and Small Startups:** Python developers building web applications, APIs, or automation tools who cannot afford enterprise SAST subscriptions. Smart SAST provides enterprise-style contextual analysis -- taint tracking, exposure detection, fix guidance -- in a zero-cost, pip-installable package that runs in under 10 seconds on a standard laptop.
- **DevSecOps Engineers at SMEs:** Security engineers integrating automated testing into CI/CD pipelines at small and medium enterprises. Smart SAST's structured JSON output, configurable min-score threshold, and non-zero exit code on Critical findings make it a drop-in security gate for GitHub Actions, GitLab CI, or Jenkins pipelines.

6.2 Market Relevance

The global application security market was valued at approximately USD 6.3 billion in 2023 and is projected to exceed USD 25 billion by 2032, driven by increasing regulatory requirements, mandatory software bill of materials (SBOM) adoption, and the shift to DevSecOps practices across software organizations of all sizes. SAST represents the largest segment within this market by developer adoption volume, as it integrates earliest in the SDLC and requires no running application infrastructure.

Python's dominance in modern software development creates a particularly large addressable market for Python-specific tools. According to Stack Overflow Developer Survey 2023, Python ranked as the most-used programming language among professional developers for the third consecutive year. Yet Python-specific SAST tooling -- particularly free, open-source tooling that

combines detection with contextual scoring and fix guidance -- remains comparatively underdeveloped.

Regulatory and policy pressure is creating institutional demand at the lower end of the market. The US NIST Secure Software Development Framework (SSDF), the EU Cyber Resilience Act, and ISO 27001:2022 all contain requirements for security testing during development. Universities are increasingly incorporating these standards into their curricula, creating demand for teaching tools that expose students to realistic SAST workflows. Smart SAST is uniquely positioned to serve this educational segment while also being production-capable for small teams.

6.3 Competitive Analysis

Table 4: Competitive Analysis -- Smart SAST vs. Existing SAST Tools

Tool	Cost	Language	Fix Hints	Risk Scoring	Noise Filter	Offline
Veracode	Enterprise (~\$25k+/yr)	Multi-language	Partial	Static labels	Yes (partial)	No
Checkmarx	Enterprise (\$\$\$)	Multi-language	Limited	Static labels	Yes (partial)	No
SonarQube	Freemium / Community	Multi-language	Yes (SonarWay)	Static labels	Partial	Yes
Bandit	Free / Open Source	Python only	No	Static labels	No	Yes
Semgrep	Free + Pro tier	Multi-language	Limited	Static labels	No	Yes
Smart SAST	Free / Open Source	Python (v1.0)	Yes (built-in)	Dynamic (0-10)	Yes (built-in)	Yes

The competitive landscape reveals that no existing free, open-source tool provides all three features that Smart SAST delivers: dynamic contextual risk scoring, built-in fix hints, and configurable noise filtering. Bandit is the closest open-source competitor for Python, but it uses only static severity labels, provides no fix guidance, and has no noise filtering mechanism. Semgrep is more flexible but requires developers to write their own rules in a custom YAML DSL and does not include fix hints by default.

Enterprise tools such as Veracode and Checkmarx include some of these features but require expensive licenses, mandate cloud connectivity, and are configured and maintained by dedicated

security teams -- making them inaccessible to students and most small startups. SonarQube Community Edition is free and includes some fix guidance but uses static severity labels and does not perform intra-procedural taint analysis for Python.

Smart SAST's differentiated value proposition is: the combination of contextual risk scoring (not available in any free tool), built-in fix hints (not available in Bandit or Semgrep), intelligent noise filtering (not available in any free tool), fully offline operation, and single-command simplicity. This combination has no direct equivalent in the current free and open-source SAST landscape for Python.

7. Gantt Chart -- Project Timeline

The table below shows the complete 14-day implementation schedule. Each two-day interval is represented by one column. Shaded cells (dark blue) indicate active work periods for each task. Team member assignments reflect individual responsibilities, though all members collaborate on testing and integration.

Task / Activity	Owner	D1-2	D3-4	D5-6	D7-8	D9-10	D11-12	D13-14
Project Setup & Dependency Installation	All							
Study Existing Tools (Bandit, Semgrep, OWASP)	All							
Repo Loader + CLI (argparse)	All							
AST Analyzer + Taint / Exposure Analysis	All							
Rule Engine -- 10 Vulnerability Classes	All							
Exploitability Scoring Engine	All							
Fix-Hint Generator (all 10 rule hints)	All							
Noise Filter (dedup + threshold suppression)	All							
Report Generator -- JSON + HTML Output	All							
Unit Testing (pytest, 8+ test cases)	All							
Integration Testing + Bug Fixes	All							
Final Report Writing	All							
Presentation & Demo Preparation	All							

Table 5: 14-Day Project Gantt Chart

7.1 Timeline Notes

- Days 1-2 (Setup): All team members configure the Python 3.10+ environment, install development dependencies (pytest, flask for testing, ast built-in), and independently study existing Python SAST tools (Bandit, Semgrep) to understand what has already been done well and where gaps exist. Muhammad Bilal leads the initial project structure and repository setup on GitHub.
- Days 3-6 (Core Pipeline): Three components are built in parallel streams. Muhammad Bilal builds the Repo Loader, AST Analyzer with taint and exposure analysis, and the Exploitability Scoring Engine. Zain Asif builds the Rule Engine, writing and testing individual rules for all 10 vulnerability classes. Faisal Rehman begins the Fix-Hint Generator and starts the Report Generator structure.
- Days 7-10 (Novel Features + Output): Zain Asif completes the Noise Filter module. Faisal Rehman completes the HTML and JSON Report Generator. Muhammad Bilal integrates all components into the end-to-end pipeline and tests the full workflow on sample vulnerable Python projects.
- Days 11-14 (Testing, Report, Presentation): All team members contribute to pytest test suite development targeting all 8+ test cases. Integration testing is performed against real Python open-source projects to identify any false positives or missed detections. Muhammad Bilal writes the final technical documentation. All members prepare the presentation and live demonstration.

8. Results and Conclusion

8.1 Expected Outcomes

As this report covers Phase 1 (planning and design), implementation has not yet begun and no experimental results are available. The following are the specific, measurable outcomes expected upon completion of the implementation phase based on the design decisions made:

- **Detection Coverage:** The Rule Engine is expected to detect at least 80% of deliberately introduced vulnerabilities in the project's sample test fixture files, covering all 10 vulnerability classes. False negative analysis will be documented for rules where precision/recall tradeoffs are significant.
- **Exploitability Score Differentiation:** For identical vulnerability patterns, findings inside public Flask/Django route functions with tainted parameters are expected to score at least 4.0 points higher (taint: +2.0, exposure: +2.0) than identical findings in test utility functions. This differentiation is the primary novel contribution of the Exploitability Scorer.
- **Noise Reduction Rate:** The de-duplication mechanism is expected to reduce displayed finding counts by 20-40% on real Python projects that contain repeated code patterns such as copy-pasted database query blocks or similar utility functions.
- **Fix-Hint Coverage:** 100% of detected findings will have an associated fix hint. This is guaranteed by design -- the Rule Engine will not produce a finding for a rule that does not have a corresponding entry in the Fix-Hint dictionary.
- **Performance:** Full analysis of a Python project with up to 10,000 lines of code is expected to complete in under 10 seconds on a standard development laptop (Intel Core i5, 8GB RAM). This ensures the tool does not slow down the development feedback loop.
- **False Positive Rate:** The Exploitability Scorer's context penalty is designed to reduce false positives in test files and dead code by assigning them lower scores, effectively suppressing them when the default min-score threshold is applied.

8.3 Conclusion

Smart SAST addresses a real, well-documented, and consequential gap in Python security tooling: the disconnect between what SAST tools can detect and what developers can actually understand and act upon. The three novel features -- Exploitability Scoring, Fix-Hint Generation, and Noise Filtering -- are each individually validated by peer-reviewed research and collectively form a coherent developer experience improvement that no existing free tool provides.

The planning phase has established a solid, research-grounded foundation for implementation. The architecture is modular and testable. The team is organized with clear responsibilities. The timeline is realistic and accounts for integration and testing overhead. The deliverables -- a working CLI tool, a test suite, JSON and HTML report outputs, and this documentation -- are clearly defined and achievable within the project scope.

Upon completion, Smart SAST will demonstrate that developer experience and security capability are not competing goals. A tool can be fast, simple, free, and locally runnable while still providing contextual risk intelligence and actionable remediation guidance. This principle -- that security tooling should work for developers, not against them -- is the core philosophical contribution of the Smart SAST project.

References

- [1] B. Chess and G. McGraw, "Static analysis for security," *IEEE Security & Privacy*, vol. 2, no. 6, pp. 76-79, Nov.-Dec. 2004, doi: 10.1109/MSP.2004.111.
- [2] B. Johnson, Y. Song, E. Murphy-Hill, and R. Bowdidge, "Why don't software developers use static analysis tools to find bugs?," in *Proc. 35th Int. Conf. Software Engineering (ICSE 2013)*, San Francisco, CA, USA, May 2013, pp. 672-681, doi: 10.1109/ICSE.2013.6606613.
- [3] C. Sadowski, J. van Gogh, C. Jaspan, E. Soderberg, and C. Winter, "Lessons from building static analysis tools at Google," *Communications of the ACM*, vol. 61, no. 4, pp. 58-66, Apr. 2018, doi: 10.1145/3188720.
- [4] N. Ayewah, D. Hovemeyer, J. D. Morgenthaler, J. Penix, and W. Pugh, "Using static analysis to find bugs," *IEEE Software*, vol. 25, no. 5, pp. 22-29, Sep.-Oct. 2008, doi: 10.1109/MS.2008.130.
- [5] V. B. Livshits and M. S. Lam, "Finding security vulnerabilities in Java applications with static analysis," in *Proc. 14th USENIX Security Symposium (USENIX Security '05)*, Baltimore, MD, USA, Aug. 2005, pp. 271-286.
- [6] S. Arzt, S. Rasthofer, C. Fritz, E. Bodden, A. Bartel, J. Klein, Y. Le Traon, D. Ochteau, and P. McDaniel, "FlowDroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for Android apps," *ACM SIGPLAN Notices (PLDI '14)*, vol. 49, no. 6, pp. 259-269, Jun. 2014, doi: 10.1145/2594291.2594299.
- [7] S. M. Smith, B. Johnson, and E. Murphy-Hill, "Questions developers ask while diagnosing potential security vulnerabilities with static analysis," in *Proc. 2015 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*, Bergamo, Italy, Sep. 2015, pp. 248-259, doi: 10.1145/2786805.2786812.

- [8] T. Muske and A. Serebrenik, "Survey of approaches for handling static analysis alarms," in Proc. 16th IEEE Int. Working Conf. Source Code Analysis and Manipulation (SCAM 2016), Raleigh, NC, USA, Oct. 2016, pp. 157-166, doi: 10.1109/SCAM.2016.25.
- [9] M. Christakis and C. Bird, "What developers want and need from program analysis: An empirical study," in Proc. 31st IEEE/ACM Int. Conf. Automated Software Engineering (ASE 2016), Singapore, Sep. 2016, pp. 332-343, doi: 10.1145/2970276.2970347.
- [10] J. Viega, J. T. Bloch, T. Kohno, and G. McGraw, "ITS4: A static vulnerability scanner for C and C++ code," in Proc. 16th Annual Conf. Computer Security Applications (ACSAC 2000), New Orleans, LA, USA, Dec. 2000, pp. 257-267, doi: 10.1109/ACSAC.2000.898880.

Appendix

Appendix A: Complete Vulnerability Rule Reference

The following table lists all ten security rules implemented in Smart SAST's Rule Engine. Each entry includes the rule ID, vulnerability class, detection method, base severity score before context adjustments, and a representative code pattern that triggers the rule.

Table A1: Smart SAST Rule Engine -- Complete Rule Reference

Rule ID	Vulnerability Class	Detection Method	Base Score	Example Triggering Pattern
sqli-001	SQL Injection	AST + Taint	4.0	<code>cursor.execute('SELECT...' + user_input)</code>
xss-001	Cross-Site Scripting	AST + Taint	3.5	<code>render_template_string(user_data)</code>
secret-001	Hardcoded Secret	Regex	3.0	<code>API_KEY = 'sk-abc123...'</code>
deser-001	Insecure Deserialization	AST	4.0	<code>pickle.loads(data)</code> / <code>yaml.load(f)</code>
crypto-001	Weak Cryptography	AST + Regex	2.5	<code>hashlib.md5(pwd)</code> / <code>DES cipher</code>
cmdi-001	Command Injection	AST + Taint	4.0	<code>subprocess.run(cmd, shell=True)</code>
path-001	Path Traversal	AST + Taint	3.5	<code>open('/files/' + user_path, 'r')</code>
rand-001	Insecure Randomness	AST	1.5	<code>random.random()</code> / <code>random.choice()</code>
debug-001	Debug / Log Leak	Regex + AST	2.5	<code>app.run(debug=True)</code> / <code>DEBUG=True</code>
file-001	Unsafe File Handling	AST	2.0	<code>open(user_filename, 'w')</code>

Appendix D: Glossary of Technical Terms

Abstract Syntax Tree (AST): A hierarchical tree representation of the grammatical structure of source code, produced by a language parser. Each node in the tree represents a construct such as a function call, variable assignment, or loop.

SAST (Static Application Security Testing): A security testing methodology that analyzes application source code, bytecode, or binary code for security vulnerabilities without executing the program.

Taint Analysis: A static analysis technique that tracks the flow of untrusted data ('tainted' data) from input sources through program logic to sensitive operations ('sinks') to detect injection vulnerabilities.

Exploitability: The degree to which a security vulnerability can realistically be leveraged by an attacker, considering factors such as attack complexity, required access level, and the vulnerability's location in the program's execution path.

Sink: In taint analysis, a function or operation that could cause harm if it receives untrusted data (e.g., `cursor.execute()`, `subprocess.run()`, `os.system()`).

Source: In taint analysis, a point where untrusted user data enters the program (e.g., `request.args.get()`, `input()`, `sys.argv`).

Noise (SAST context): False positive findings or low-impact alerts that do not represent actionable security risks but clutter the tool's output, contributing to alert fatigue.

CVE (Common Vulnerabilities and Exposures): A standardized identifier for publicly known software security vulnerabilities, maintained by MITRE Corporation and used across the security industry.

OWASP Top 10: A regularly updated list of the ten most critical web application security risks, maintained by the Open Web Application Security Project (OWASP) and used as a baseline security standard worldwide.

CI/CD: Continuous Integration / Continuous Deployment -- automated pipelines that build, test, and deploy software changes in a controlled, repeatable manner, enabling fast and reliable software delivery.